Distributed LLM Training — Run Plan

Llama-3-8B · 4× A100 SXM 80GB · RunPod · MLflow tracking

Stage 1 — Single GPU — Peak Memory Experiment

Goal: measure actual HBM needed at any moment to decide which GPU tier fits each configuration.

Run 1	Gradient checkpointing ON python train_single.py
Run 2	Gradient checkpointing OFF (comment out gradient_checkpointing_enable(), change run_name)
Metrics logged	samples_per_sec · steady_memory_mb · peak_memory_mb
Key formula	peak(OFF) – peak(ON) = activation memory saved by checkpointing savings % = (peak(OFF) – peak(ON)) / peak(OFF) × 100%
Decision	peak_memory_mb = minimum GPU HBM needed if peak(ON) fits a cheaper GPU → checkpointing pays off cost = 18% throughput penalty

Why peak and not steady-state:
Steady-state (after optimizer.step) only shows weights + gradients + optimizer states — activations already freed. OOMs happen during the forward→backward window. Peak memory right after backward() is the true ceiling.

Stage 2 — DDP — Throughput Scaling

Goal: measure throughput gains and communication overhead as GPUs increase. Single GPU throughput from Stage 1 is the baseline.

Commands	torchrun --nproc_per_node=2 train_ddp.py torchrun --nproc_per_node=4 train_ddp.py (1 GPU skipped — identical to single GPU baseline)
Batch size	Fixed at 4 across all runs (clean comparison)
Metrics logged	samples_per_sec · peak_memory_mb per rank
Key formula	scaling efficiency = (N_gpu_throughput / (N × 1gpu_throughput)) × 100% target: 80–92% at 4 GPUs on NVLink
Memory expectation	Peak per rank = same as single GPU DDP does not shard — full model on every rank
Throughput expectation	~1.8× at 2 GPUs, ~3.4× at 4 GPUs (not linear — all-reduce communication overhead)

DDP answers: how much throughput do you gain by adding GPUs? Memory problem unchanged.

Stage 3 — FSDP — Memory Savings + Throughput Recovery

Goal: show memory drop from sharding. Then use freed memory for larger batches to recover throughput.

Commands	torchrun --nproc_per_node=4 train_fsdp.py (batch=4) torchrun --nproc_per_node=4 train_fsdp.py (batch=16) torchrun --nproc_per_node=2 train_fsdp.py (batch=16)
Gradient checkpointing	OFF — sharding alone should be enough on A100 80GB
Metrics logged	samples_per_sec · peak_memory_mb · steady_memory_mb
Memory expectation	Peak per rank dramatically lower than DDP FSDP shards weights + gradients + optimizer states across 4 GPUs
Throughput story	FSDP 4GPU batch=4 → lower than DDP (communication overhead) FSDP 4GPU batch=16 → back to ~DDP level (bigger batch amortizes cost) FSDP 2GPU batch=16 → can 2 GPUs match DDP 4 GPU throughput?
Key insight	DDP is memory-constrained on batch size. FSDP frees memory → bigger batch → recovers throughput lost to communication. FSDP 2 GPUs ≈ DDP 4 GPUs in throughput = same job, half the GPU cost.

Stage 4

Ray Train — Managed DDP

Goal: same throughput as DDP with simpler setup. No manual process group init or MASTER_ADDR config.

Command	python train_ray.py
Memory expectation	Same as DDP — Ray Train wraps DDP, same memory math
Throughput expectation	Same as DDP — same underlying communication
What changes	No manual dist.init_process_group No MASTER_ADDR/PORT setup Native HPO integration via Ray Tune
Decision	Same numbers, simpler setup. Worth it for large clusters where multi-node coordination is the hard part.

Expected Results Summary

Stage	GPUs	Batch	Peak Mem/rank	Throughput	Key finding
Single GPU (ckpt ON)	1	4	TBD	4.61 s/s	Baseline
Single GPU (ckpt OFF)	1	4	TBD	5.45 s/s	peak diff = activation cost
DDP	2	4	~same as single	TBD	scaling efficiency at 2 GPUs
DDP	4	4	~same as single	TBD	scaling efficiency at 4 GPUs
FSDP	4	4	TBD (much lower)	TBD	apples-to-apples vs DDP
FSDP	4	16	TBD (much lower)	TBD	recover throughput with freed memory
FSDP	2	16	TBD (much lower)	TBD	2 GPUs ≈ DDP 4 GPUs?
Ray Train	4	4	~same as DDP	TBD	setup simplicity vs torchrun

Tracking Results — compare_runs.py

After each stage, run compare_runs.py on the pod to auto-calculate scaling efficiency and peak memory savings. Reads all runs from MLflow — runs not yet completed show TBD.

Run on pod	python compare_runs.py (MLflow server must be running at localhost:8080)
Memory table	peak_memory_mb · steady_memory_mb · activation_memory per run checkpointing savings % = (peak_OFF – peak_ON) / peak_OFF × 100%
Scaling table	DDP and FSDP only expected = single_gpu_throughput × world_size actual multiplier = run_throughput / single_gpu_throughput efficiency % = (actual / expected) × 100%
Partial runs OK	Script works at any stage — missing runs print TBD Run after single GPU → after DDP → after FSDP to see table fill progressively

GPU sizing decision from peak memory:
peak(checkpointing OFF) = minimum GPU HBM without tricks. peak(checkpointing ON) = minimum GPU HBM with checkpointing. If the cheaper GPU tier fits peak(ON), checkpointing pays off — cost is 18% throughput.